SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Ch.Ishwarya	**Reg. No.:**	192324145
Section / Batch:	AI&DS(2023)	**Date:**	12-10-26

Code of Conduct

I Ch.Ishwarya(192324145) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ch.Ishwarya

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Question 1: Resume Information Extraction System

Problem Statement

Design and implement a Python-based resume information extraction system using Regular Expressions that extracts the candidate's name, email addresses, mobile numbers, technical skills (Python, Java, SQL, Machine Learning, NLP) and years of experience, generates a structured profile summary, and displays candidates who satisfy the minimum eligibility criteria (at least 2 years of experience and Python skill).

Approach

- Each field is extracted by its own small function using a pre-compiled regular expression (modular design).
- `build_profile()` combines all extractors into one structured dictionary per resume.
- `is_eligible()` applies the shortlisting rule: experience >= 2 years AND "Python" in the skill list.
- Mobile numbers are normalised to plain 10-digit form; empty or invalid resume input raises a `ValueError`.

Regular Expression Patterns Used

Field	Pattern	Explanation
Name	`^\s*(?:Name\s*[:\-]\s*)?([A-Z][a-z]+(?:\s+[A-Z][a-z]*\.?){1,3})\s*$`	Optional "Name:" label followed by 2-4 capitalised words on a line of their own
Email	`\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b`	username, @, domain, dot, TLD of 2+ letters
Mobile	`(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{4}[\s-]?\d{5})(?!\d)`	Indian mobile: optional +91 / 0, first digit 6-9, 10 digits in total
Skills	`\bSkill\b (re.IGNORECASE)`	Word boundary stops "Java" matching inside "JavaScript"
Experience	`(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b`	Integer or decimal number followed by year / years / yr / yrs

Python Program

```
    """
    Q1 - Resume Information Extraction System (Regular Expressions)
    """
    import re

    # ---------- Compiled regular expression patterns ----------
    NAME_PATTERN        =        re.compile(r"^\s*(?:Name\s*[:\-]\s*)?([A-Z][a-z]+(?:\s+[A-Z][a-z]*\.?){1,3})\s*$",
    re.MULTILINE)
    EMAIL_PATTERN = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
    MOBILE_PATTERN = re.compile(r"(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{4}[\s-]?\d{5})(?!\d)")
    EXP_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b", re.IGNORECASE)

    SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
    # \b prevents "Java" matching inside "JavaScript"
    SKILL_PATTERNS = {s: re.compile(r"\b" + re.escape(s) + r"\b", re.IGNORECASE) for s in SKILLS}

def extract_name(text):
        m     =     NAME_PATTERN.search(text)     return
        m.group(1).strip() if m else "Not found"

def extract_emails(text):
        return EMAIL_PATTERN.findall(text)
def extract_mobiles(text):
        # normalise to a plain 10-digit number
        return [re.sub(r"[\s-]", "", n) for n in MOBILE_PATTERN.findall(text)]

def extract_skills(text):
        return [skill for skill, pat in SKILL_PATTERNS.items() if pat.search(text)]

def extract_experience(text):
        years  =  [float(y)  for  y  in  EXP_PATTERN.findall(text)]
        return max(years) if years else 0.0

def build_profile(resume_text):
        """Return a structured dictionary summarising one resume."""
        if     not     isinstance(resume_text,     str)     or     not
        resume_text.strip():
            raise ValueError("Resume text is empty or invalid")
        return {
            "name": extract_name(resume_text),
            "emails": extract_emails(resume_text),
            "mobiles": extract_mobiles(resume_text),
            "skills": extract_skills(resume_text),
            "experience": extract_experience(resume_text),
        }

def is_eligible(profile, min_exp=2, required_skill="Python"):
        return profile["experience"] >= min_exp and required_skill in profile["skills"]
```

```
 def   print_profile(profile):
    print("-" * 55)
        print(f"Name       : {profile['name']}")
        print(f"Email      : {', '.join(profile['emails']) or 'Not found'}")
        print(f"Mobile     : {', '.join(profile['mobiles']) or 'Not found'}")
        print(f"Skills     :    {',     '.join(profile['skills'])    or    'None
                           detected'}")
        print(f"Experience       :       {profile['experience']:g}       year(s)")
        print(f"Eligible : {'YES' if is_eligible(profile) else 'NO'}")

if   __name__==  "__main__":
    resumes = [
            """Rahul Sharma
    Email: rahul.sharma@gmail.com
    Mobile: +91 98765 43210
    Skills: Python, SQL, Machine Learning, NLP
    Experience: 3 years as Data Scientist""",
            """Name: Priya Nair
    Contact: priya.nair@outlook.com | 9123456780
    Technical Skills: Java, SQL, JavaScript
    Total experience of 5 years in backend development""",
            """Arun Kumar
    arun_k@yahoo.co.in
    Phone:    91234-56789
    Skills: python, nlp
    Fresher with 1 year internship experience""",
            """Sneha                          Reddy
    sneha.reddy@company.org
    +91-8012345678
    Expertise: Python, Java, Machine Learning
    Experience: 2.5 yrs in AI development""",
        ]
profiles = [] for idx, resume in enumerate(resumes,
        start=1):
            print(f"\nCANDIDATE      {idx}")
            try:
                profile = build_profile(resume)
                profiles.append(profile)
                print_profile(profile)
            except      ValueError      as      err:
                print("Error:",  err)  print("\n"  +
                "="    *    55)    print("SHORTLISTED
                CANDIDATES  (>=  2  years  experience
                AND   Python)")   print("="   *   55)
                shortlisted = [p for p in profiles
                if   is_eligible(p)]   for   p   in
                shortlisted:
          print(f"* {p['name']:<15} | {p['experience']:g} yrs | {', '.join(p['skills'])}")
        print(f"\nTotal shortlisted: {len(shortlisted)} of {len(profiles)}")

        #     Error-handling     test
        print("\nEmpty-resume
        test:") try:
            build_profile(" ")
        except      ValueError      as      err:
            print("Error:", err)
```

Output

```
    CANDIDATE 1
    ---------------------------------------------------------

    Name       : Rahul Sharma
    Email      : rahul.sharma@gmail.com
    Mobile     : 9876543210
    Skills     : Python, SQL, Machine Learning, NLP
    Experience : 3 year(s)
    Eligible : YES

    CANDIDATE 2
    ---------------------------------------------------------


    Name       : Priya Nair
```

```
Email     : priya.nair@outlook.com

Mobile    : 9123456780
Skills : Java, SQL Experience :
5 year(s)
Eligible : NO

CANDIDATE 3
_____________________________________________________


Name       : Arun Kumar
Email      : arun_k@yahoo.co.in
Mobile     : 9123456789
Skills     : Python, NLP
Experience : 1 year(s)
Eligible : NO

CANDIDATE 4
_____________________________________________________


Name       : Sneha Reddy
Email      : sneha.reddy@company.org
Mobile     : 8012345678
Skills     : Python, Java, Machine Learning
Experience : 2.5 year(s)
Eligible : YES

=======================================================
SHORTLISTED CANDIDATES (>= 2 years experience AND Python)
=======================================================
* Rahul Sharma     | 3 yrs | Python, SQL, Machine Learning, NLP
* Sneha Reddy      | 2.5 yrs | Python, Java, Machine Learning

Total shortlisted: 2 of 4

Empty-resume test:
Error: Resume text is empty or invalid
```

Test Cases and Justification

Test Case	Input Highlights	Result / Justification
Candidate 1	Python, SQL, ML, NLP; 3 years; +91 98765 43210	Shortlisted. Experience 3 >= 2 and Python present. The +91 and spaces are stripped to give 9876543210.
Candidate 2	Java, SQL, JavaScript; 5 years	Not shortlisted. Experience is enough but Python is missing. JavaScript is correctly not counted as Java thanks to \b.
Candidate 3	python, nlp (lower case); 1 year	Not shortlisted. Lower-case skills are still detected (IGNORECASE) but 1 year < 2 years.
Candidate 4	Python, Java, ML; 2.5 yrs	Shortlisted. The decimal 2.5 and abbreviation "yrs" are handled by the experience pattern.
Empty resume	" " (blank)	ValueError raised and displayed, so the program does not crash.

Conclusion: 2 of 4 candidates (Rahul Sharma and Sneha Reddy) satisfy the eligibility criteria. The output matches the expected result for every test case, which confirms that the extraction and filtering logic is correct.

Question 2: Product Search System

Problem Statement

Design and implement a Python-based product search system using Regular Expressions that supports exact, prefix, suffix, partial and case-insensitive searches, displays all matching product names, and generates a report with the number of matches for each search.

Approach

- `build_pattern()` creates the regex for the chosen mode; `search()` compiles it and filters the product list.
- `re.escape()` is applied to the keyword so special characters typed by the user are matched literally and cannot break the pattern.
- Case-insensitivity is controlled by a flag (`re.IGNORECASE`), so the same patterns work for both cases.
- `run_searches()` prints the matches and collects counts; `print_report()` displays the final summary table.

Regular Expression Patterns Used

Search Type	Pattern	Meaning
Exact	`^keyword$`	Whole product name must equal the keyword
Prefix	`\bkeyword\w*`	A word in the name begins with the keyword (e.g. Smart -> Smartphone)
Suffix	`\w*keyword\b`	A word in the name ends with the keyword (e.g. phone -> Headphone)
Partial	`keyword`	Keyword appears anywhere in the name
Case-insensitive	`re.IGNORECASE flag`	Upper and lower case are treated as the same

Python Program

```
"""
Q2 - Product Search System (Regular Expressions)
"""
import re

PRODUCTS = [
    "Laptop", "Laptop Bag", "Gaming Laptop", "Wireless Mouse", "Mouse Pad",
    "Mechanical Keyboard", "Keyboard Cover", "Smartphone", "Smartwatch",
    "Phone Case", "Headphones", "Bluetooth Headphone", "Earphones",
    "USB Cable", "USB Charger", "Power Bank", "Laptop Stand", "Webcam",
    "Tablet", "Tablet Cover",
]

def build_pattern(keyword, mode):
    """Return the regex string for the requested search mode."""
    if not keyword or not keyword.strip():
        raise ValueError("Search keyword cannot be empty")
    kw = re.escape(keyword.strip()) patterns # treat user input literally
    = {
        "exact": rf"^{kw}$",                # whole product name equals keyword
        "prefix": rf"\b{kw}\w*",            # a word starting with keyword
        "suffix": rf"\w*{kw}\b",            # a word ending with keyword
        "partial": rf"{kw}",                # keyword anywhere
    }   if   mode   not   in
    patterns:
        raise ValueError(f"Unknown search mode: {mode}")
    return patterns[mode]

def search(products, keyword, mode="partial", ignore_case=True):
    flags = re.IGNORECASE if ignore_case else 0 pattern =
```

```
    re.compile(build_pattern(keyword, mode), flags) return [p for p
    in products if pattern.search(p)]

def run_searches(products, queries):
        report   =   []   for   keyword,   mode,
        ignore_case in queries:
            label   =   f"{mode.upper()}   |   keyword='{keyword}'   |   case-{'insensitive'   if   ignore_case   else
            'sensitive'}" print("\n" + label) try:
                matches = search(products, keyword, mode, ignore_case)
            except   ValueError   as   err:
                print("    Error:",    err)
                report.append((label,
                None)) continue
            if matches:
                for m in matches:
                    print(" -", m) else:
                print(" No matching products found")
            report.append((label, len(matches)))
        return report

def print_report(report): print("\n" + "=" *
    70)  print("SEARCH  REPORT")  print("="  *
    70)
    print(f"{'Search':<55}{'Matches':>8}")
    print("-"  *  70)  for  label,  count  in
    report:
            print(f"{label:<55}{('ERR' if count is None else count):>8}")


    if _name_== "_main_":
        queries = [
            ("Laptop", "exact", False),    # exact keyword
            ("laptop", "exact", True),     # exact, case-insensitive
            ("Smart", "prefix", True),     # prefix
            ("Cover", "suffix", True),     # suffix
            ("phone", "suffix", True),     # suffix (word ends with 'phone')
            ("board", "partial", True),    # partial keyword
            ("LAPTOP", "partial", False), # case-sensitive -> 0 matches ("LAPTOP",
            "partial", True), # case-insensitive -> matches
            ("", "partial", True),    # error handling
            ("xyz", "partial", True),   # no result
        ]
        print_report(run_searches(PRODUCTS, queries))
```

Output

```
    EXACT | keyword='Laptop' | case-sensitive
      - Laptop

    EXACT | keyword='laptop' | case-insensitive
      - Laptop

    PREFIX | keyword='Smart' | case-insensitive
      - Smartphone
      - Smartwatch

    SUFFIX | keyword='Cover' | case-insensitive
      - Keyboard Cover
      - Tablet Cover

    SUFFIX | keyword='phone' | case-insensitive
      - Smartphone
      - Phone Case
      - Bluetooth Headphone

    PARTIAL | keyword='board' | case-insensitive
      - Mechanical Keyboard
      - Keyboard Cover

    PARTIAL | keyword='LAPTOP' | case-sensitive
      No matching products found
```

```
PARTIAL | keyword='LAPTOP' | case-insensitive
  - Laptop
  - Laptop Bag
  - Gaming Laptop
  - Laptop Stand

PARTIAL | keyword='' | case-insensitive
  Error: Search keyword cannot be empty

PARTIAL | keyword='xyz' | case-insensitive
  No matching products found

======================================================================
SEARCH REPORT
======================================================================
Search                                                          Matches
----------------------------------------------------------------------
EXACT | keyword='Laptop' | case-sensitive                          1
EXACT | keyword='laptop' | case-insensitive                        1
PREFIX | keyword='Smart' | case-insensitive                        2
SUFFIX | keyword='Cover' | case-insensitive                        2
SUFFIX | keyword='phone' | case-insensitive                        3
PARTIAL | keyword='board' | case-insensitive                       2
PARTIAL | keyword='LAPTOP' | case-sensitive                        0
PARTIAL | keyword='LAPTOP' | case-insensitive                      4
PARTIAL | keyword='' | case-insensitive                             ERR
PARTIAL | keyword='xyz' | case-insensitive                         0
```

Test Cases and Justification

Search	Matches	Justification
Exact "Laptop"	1	Only the product named exactly "Laptop"; Laptop Bag, Gaming Laptop, etc. are excluded by ^ and $.
Prefix "Smart"	2	Smartphone and Smartwatch begin with "Smart".
Suffix "Cover"	2	Keyboard Cover and Tablet Cover end with "Cover".
Suffix "phone"	3	Smartphone, Phone Case (case-insensitive) and Bluetooth Headphone. "Headphones" and "Earphones" are not counted because the word ends with "s", not "phone".
Partial "board"	2	Found inside Keyboard in both Mechanical Keyboard and Keyboard Cover.
"LAPTOP" case-sensitive	0	Capitals do not match the stored "Laptop", which proves case-sensitive mode works.
"LAPTOP" case-insensitive	4	With IGNORECASE all four laptop-related products are found.
Empty keyword	Error	ValueError is caught and shown instead of crashing.
"xyz"	0	A clear "No matching products found" message is displayed.

Conclusion: All search types return the expected products and the report shows the correct count for every search, so the system meets all the stated requirements.

Question 3: University Registration Validation System

Problem Statement

Design and implement a Python-based validation system using Regular Expressions that validates the student register number, institutional email, course code, semester and mobile number, displays a validation message for each field, and produces a final registration status report.

Approach

- All rules are stored in one dictionary (field -> compiled pattern + format hint), so a rule can be changed without touching the logic.
- `re.fullmatch()` is used so the whole input must follow the format (no extra characters before or after).
- `validate_field()` returns (True/False, message); `validate_registration()` checks all fields; `print_report()` shows PASS/FAIL per field.
- Registration is successful only if every field is valid. Empty values are detected and reported separately.

Regular Expression Patterns Used

Field	Pattern	Valid Format
Register Number	`19(?:2[0-5])\d{5}`	9 digits: institute code 19 + year 20-25 + 5-digit serial (e.g. 192311234)
Institutional Email	`[A-Za-z0-9._%+-]+@(?:saveetha\.com|simats\.edu\.in)`	Username with an institutional domain only
Course Code	`[A-Z]{3}\d{2}`	3 capital letters + 2 digits (e.g. DSA03)
Semester	`[1-8]`	Single digit 1 to 8
Mobile Number	`(?:\+91[\s-]?)?[6-9]\d{9}`	10 digits starting with 6-9, optional +91

Note: The register number and domain formats are assumed typical institutional formats because the question does not specify them. They can be changed in the `RULES` dictionary.

Python Program

```
    """
    Q3 - University Registration Validation System (Regular Expressions)
    """
    import re

    # ---------- Validation rules (assumed institutional formats) ---------
    RULES = {
        # 9 digits: '19' institute code + 2-digit year (20-25) + 5-digit serial e.g. 192311234
        "Register Number": (re.compile(r"19(?:2[0-5])\d{5}"),
                            "9 digits, e.g. 192311234"),
        # username @ saveetha.com or simats.edu.in
        "Institutional Email": (re.compile(r"[A-Za-z0-9._%+-]+@(?:saveetha\.com|simats\.edu\.in)", re.IGNORECASE),
                                "must end with @saveetha.com or @simats.edu.in"),
        # 3 capital letters + 2 digits e.g. DSA03
        "Course Code": (re.compile(r"[A-Z]{3}\d{2}"),
                        "3 capital letters + 2 digits, e.g. DSA03"),
        # semester 1 to 8
        "Semester": (re.compile(r"[1-8]"),
                     "a single digit from 1 to 8"),
        # Indian mobile: optional +91, starts with 6-9, 10 digits
        "Mobile Number": (re.compile(r"(?:\+91[\s-]?)?[6-9]\d{9}"),
                          "10 digits starting with 6-9, optional +91"),
    }
def validate_field(field, value):
    """Return (is_valid, message)
    for one field using
    re.fullmatch.""" pattern, hint =
    RULES[field] if value is None or
    not str(value).strip(): return
    False, f"INVALID - field is
    empty ({hint})"
        if pattern.fullmatch(str(value).strip()):
            return True, "VALID"
        return False, f"INVALID - {hint}"

def validate_registration(student):
        results = {} for field in RULES: results[field] =
        validate_field(field, student.get(field))
        return results
```

```
def print_report(student, results):
        print("-" * 66) for field, (ok, msg) in
        results.items():
            mark = "[PASS]" if ok else "[FAIL]" print(f"{mark} {field:<20}:
            {student.get(field)!s:<24} -> {msg}")
        status = all(ok for ok, _ in results.values()) print("-"
        * 66)
        print("REGISTRATION  STATUS:",  "SUCCESSFUL"  if  status  else  "REJECTED")
        return status

if _name_== "_main_":
        students = [
            ("Test Case 1 - All valid", {
                "Register Number": "192311234", "Institutional Email": "rahul.s@saveetha.com",
                "Course Code": "DSA03", "Semester": "5", "Mobile Number": "+91 9876543210"}),
            ("Test Case 2 - Multiple errors", {
                "Register Number": "12345", "Institutional Email": "rahul@gmail.com",
                "Course Code": "dsa3", "Semester": "9", "Mobile Number": "1234567890"}),
            ("Test Case 3 - One error (semester)", {
                "Register Number": "192411567", "Institutional Email": "priya_n@simats.edu.in",
                "Course Code": "CSE45", "Semester": "0", "Mobile Number": "9123456780"}),
            ("Test Case 4 - Empty fields", {
                "Register Number": "", "Institutional Email": "", "Course Code": "DSA03",
                "Semester": "", "Mobile Number": "98765"}),
        ]
 summary  =  []  for  title,  data  in
        students:
            print("\n"    +    title)    summary.append((title,    print_report(data,
            validate_registration(data))))
  print("\n"    +    "="    *    66)
        print("FINAL    SUMMARY")
        for    title,    ok    in
        summary:
            print(f"{title:<40}: {'SUCCESSFUL' if ok else 'REJECTED'}")
```

Output

```
Test Case 1 - All valid
------------------------------------------------------------------------------------
[PASS] Register Number  : 192311234                  -> VALID
[PASS] Institutional Email : rahul.s@saveetha.com    -> VALID
[PASS] Course Code    : DSA03                        -> VALID
[PASS] Semester           : 5                        -> VALID
[PASS] Mobile Number   : +91 9876543210              -> VALID
------------------------------------------------------------------------------------
REGISTRATION STATUS: SUCCESSFUL

Test Case 2 - Multiple errors
------------------------------------------------------------------------------------
[FAIL] Register Number     : 12345                  -> INVALID - 9 digits, e.g. 192311234

[FAIL] Institutional Email : rahul@gmail.com        -> INVALID - must end with @saveetha.com or @simats.edu.in

[FAIL] Course Code         : dsa3                    -> INVALID - 3 capital letters + 2 digits, e.g. DSA03

[FAIL] Semester            : 9                       -> INVALID - a single digit from 1 to 8

[FAIL] Mobile Number       : 1234567890              -> INVALID - 10 digits starting with 6-9, optional +91
------------------------------------------------------------------------------------
REGISTRATION STATUS: REJECTED

Test Case 3 - One error (semester)
------------------------------------------------------------------------------------
[PASS] Register Number  : 192411567                  -> VALID
[PASS] Institutional Email : priya_n@simats.edu.in   -> VALID
[PASS] Course Code    : CSE45                        -> VALID
[FAIL] Semester           : 0                        -> INVALID -
[PASS] Mobile Number   : 9123456780                  -> VALID
------------------------------------------------------------------------------------
REGISTRATION STATUS: REJECTED
------------------------------------------------------------------------------------
```

```
--------------------------------------------------------------------------------
Test Case 4 - Empty fields
--------------------------------------------------------------------------------
[FAIL] Register Number    :                          -> INVALID -
[FAIL] Institutional Email :                         -> INVALID -
[PASS] Course Code        : DSA03                    -> VALID
[FAIL] Semester           :                          -> INVALID -
[FAIL] Mobile Number      : 98765                    -> INVALID -
--------------------------------------------------------------------------------
                                                                  a single digit from 1 to 8



field is empty (9 digits, e.g. 192311234)  field is empty (must end with @saveetha.com or @simats.ed

                                                                  field is empty (a single digit from 1 to 8)
     10 digits starting with 6-9, optional +91


    REGISTRATION STATUS: REJECTED

    ================================================================
    FINAL SUMMARY

    Test Case 1 - All valid                : SUCCESSFUL
    Test Case 2 - Multiple errors : REJECTED Test Case 3 -
    One error (semester)  : REJECTED

    Test Case 4 - Empty fields             : REJECTED
```

Test Cases and Justification

Test Case	Status	Justification
1 - All valid	SUCCESSFUL	Every field matches its pattern; +91 and the space are allowed in the mobile number.
2 - Multiple errors	REJECTED	Register number too short, Gmail domain, lowercase course code, semester 9 and mobile starting with 1 all fail.
3 - One error	REJECTED	Four fields are valid but semester 0 is outside 1-8, so a single failure rejects the registration.
4 - Empty fields	REJECTED	Empty inputs get a specific "field is empty" message; the short mobile number 98765 also fails.

Conclusion: The validator accepts only correctly formatted data, gives a clear message for each field, and the final report correctly marks only Test Case 1 as successful.

Part B: Graphical User Interface (GUI) Implementation

Overview

The three systems above were also combined into a single desktop application using Python's built-in Tkinter library. The GUI uses exactly the same regular expressions and validation logic; only the input and output are moved from the console to windows, buttons and tables, which makes the system easier to use for HR staff, customers and students.

GUI Design

- A tabbed window (`ttk.Notebook`) holds one tab for each question.

- **Tab 1 - Resume Extraction:** a text box for resumes, an Extract button, a results table with eligible candidates highlighted green, and a shortlist summary.
- **Tab 2 - Product Search:** keyword box, search-type drop-down (Exact / Prefix / Suffix / Partial), an Ignore-case check box, a results list and a running search report table.
- **Tab 3 - Registration Validation:** a form with five fields, a tick or cross beside each field, a coloured SUCCESSFUL / REJECTED banner and a detailed report.
- **Error handling:** message boxes for empty input or an empty keyword; invalid form data is reported field by field.

GUI Screenshots

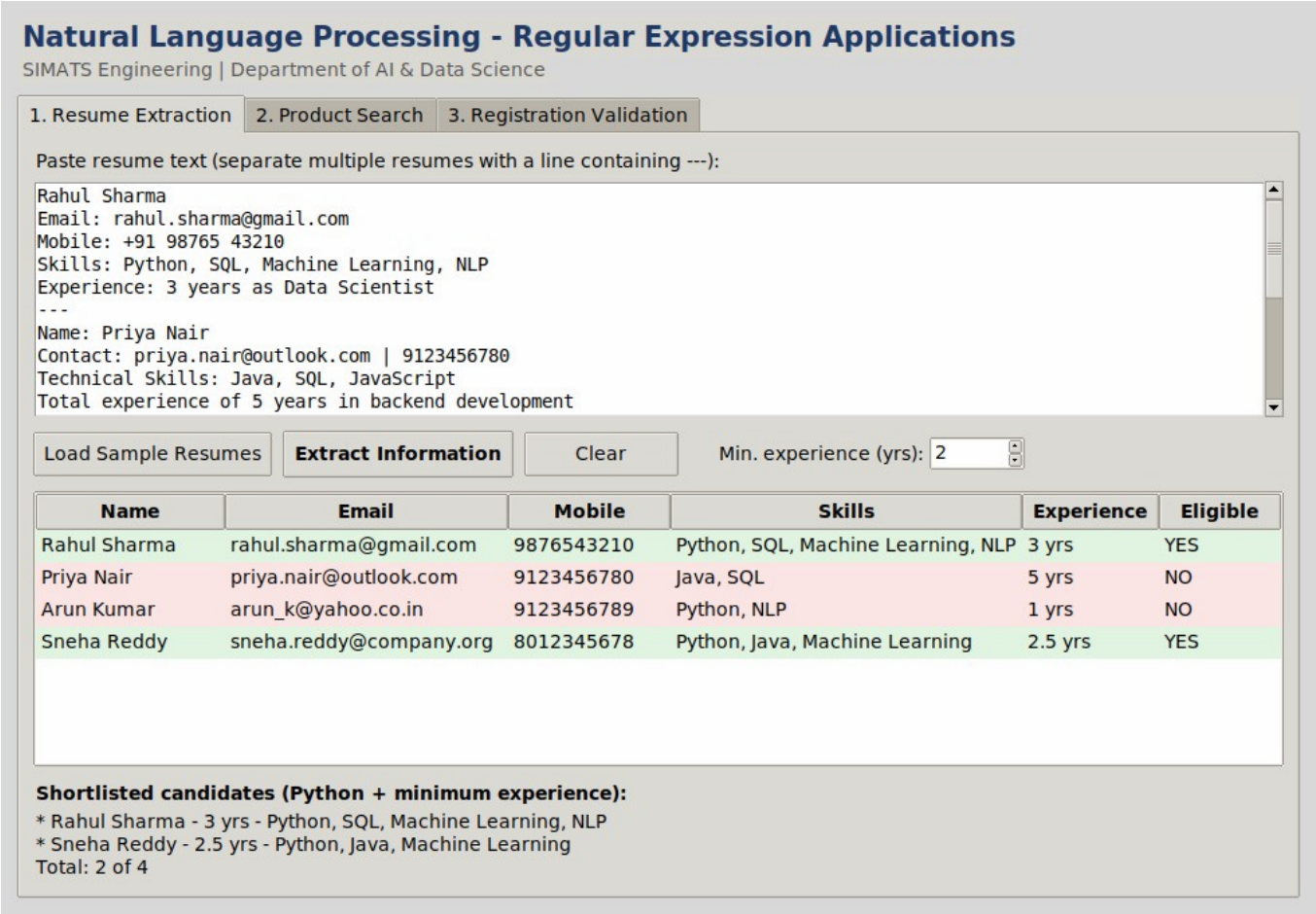


Figure 1: Resume Extraction tab - 2 of 4 candidates shortlisted

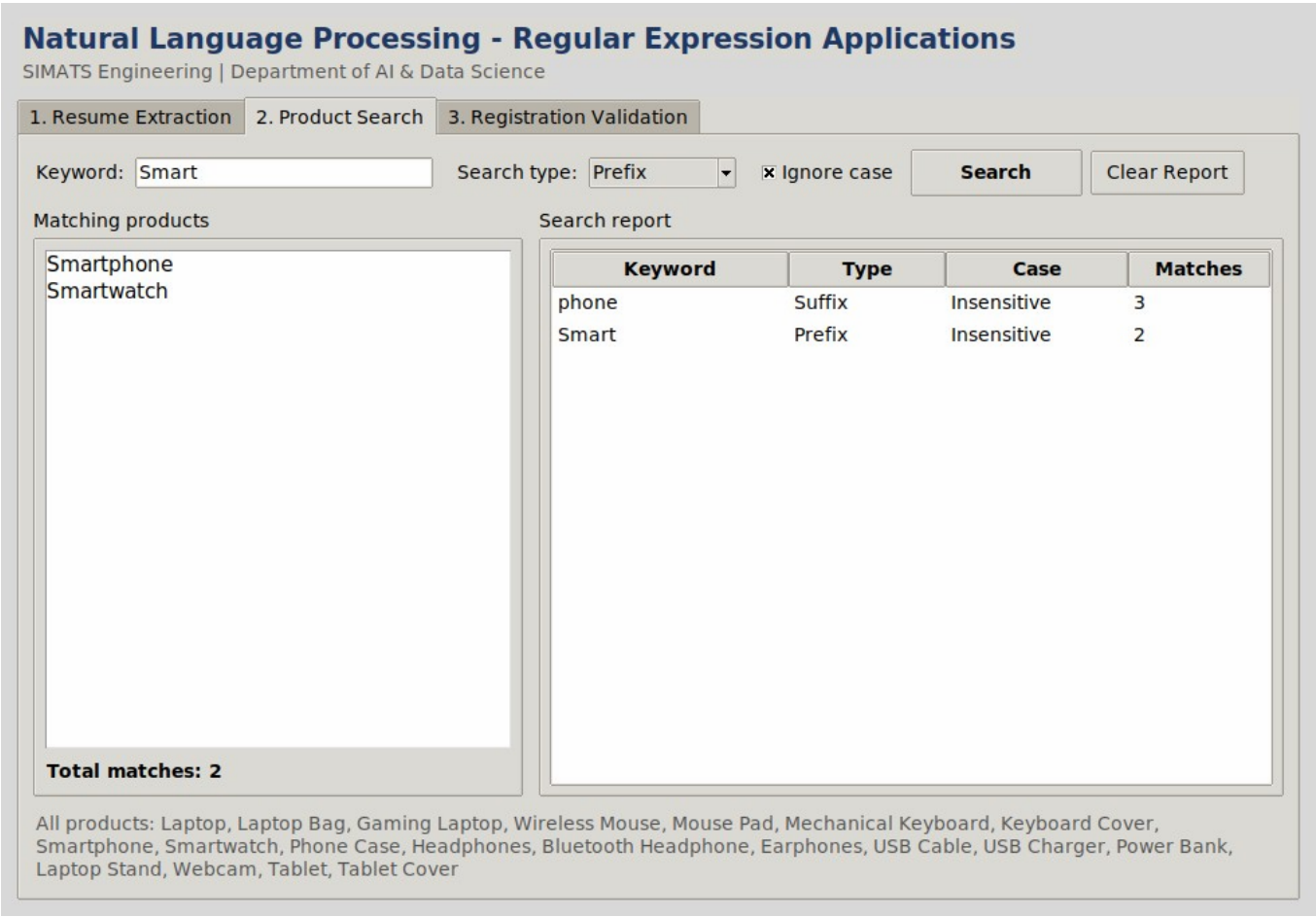


Figure 2: Product Search tab - prefix and suffix searches with report

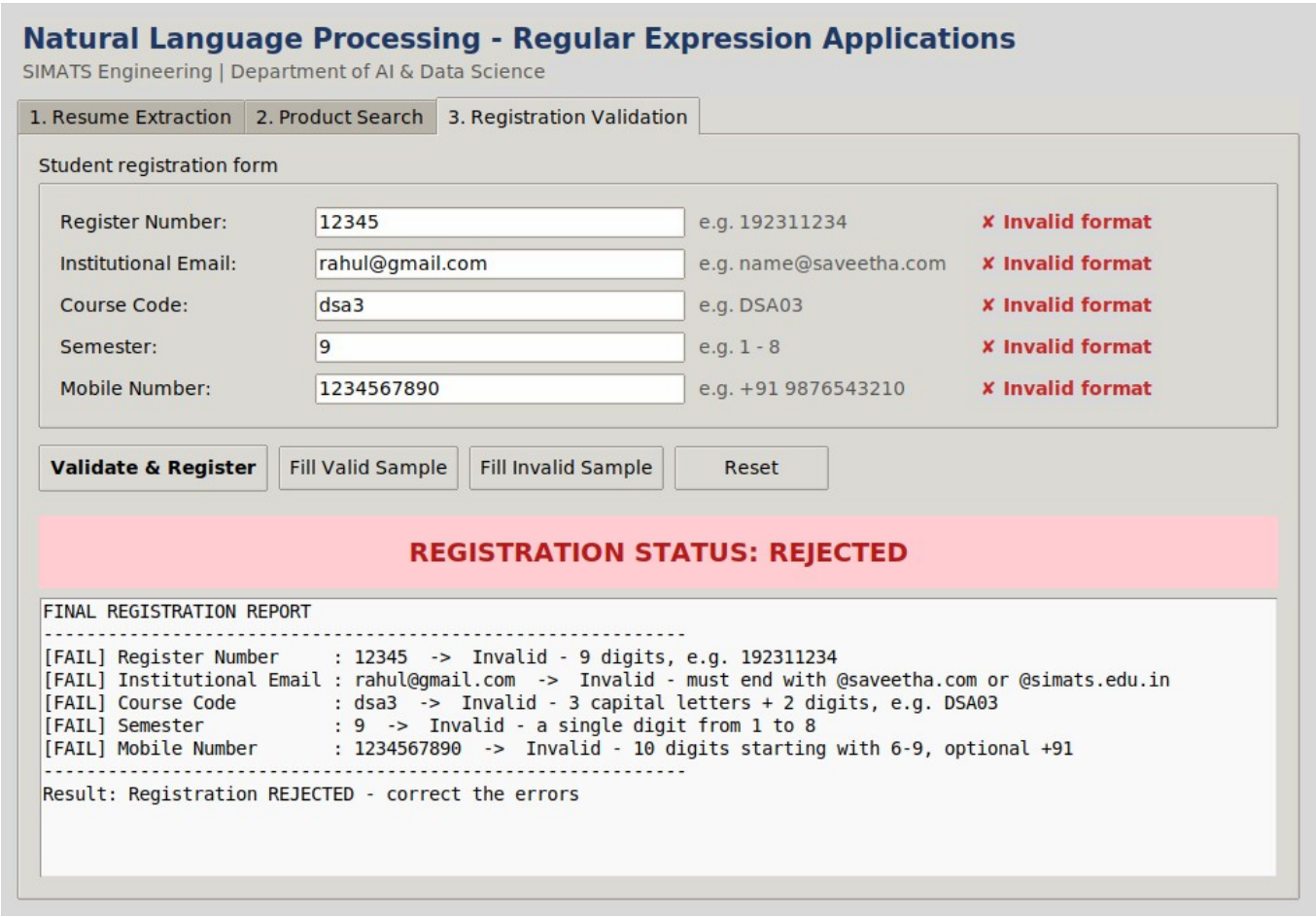


Figure 3: Registration Validation tab - invalid data rejected

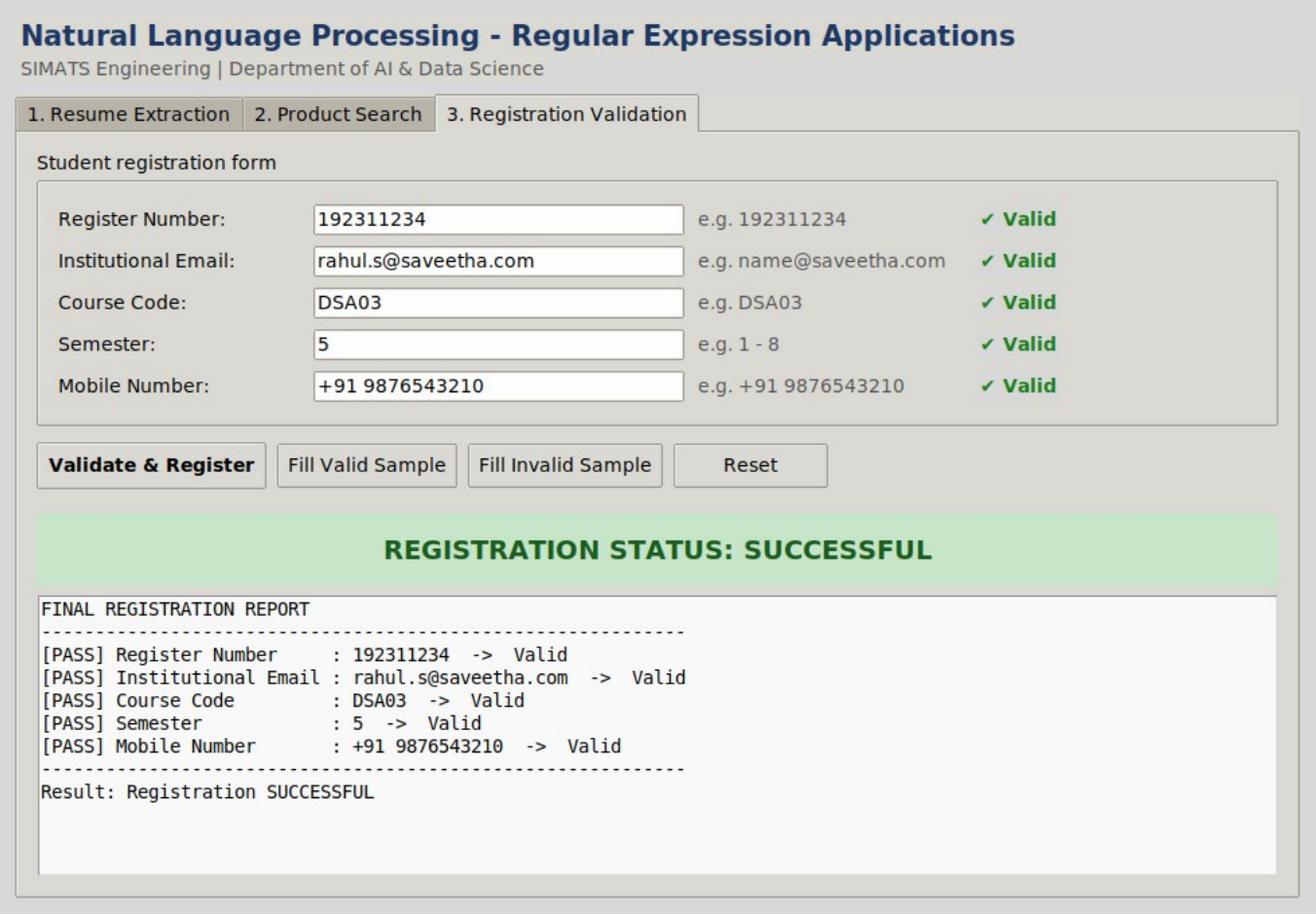


Figure 4: Registration Validation tab - valid data accepted

Test Cases and Justification

GUI Action	Input	Observed Result
Load Sample + Extract	4 sample resumes	Table shows 4 candidates; Rahul Sharma and Sneha Reddy highlighted green (shortlisted), total 2 of 4.
Product search	Keyword "phone", Suffix, Ignore case	3 matches (Smartphone, Phone Case, Bluetooth Headphone) added to the report.
Product search	Keyword "Smart", Prefix	2 matches (Smartphone, Smartwatch); the report table now has two rows.
Fill Invalid Sample + Validate	All five fields incorrect	Each field marked with a red cross; banner shows REJECTED.
Fill Valid Sample + Validate	All five fields correct	Each field marked with a green tick; banner shows SUCCESSFUL.

Conclusion: The GUI gives the same results as the console programs for all test cases, confirming that the regular-expression logic is correct and that the interface presents it clearly.

GUI Python Program

Run: `python nlp_gui.py`

```
"""
DSA03 - NLP | CO1 Assessment Tool 1
GUI application (Tkinter) for:
```

```
      Tab 1 - Resume Information Extraction
      Tab 2 - Product Search System
      Tab 3 - University Registration Validation
    Run: python nlp_gui.py
    """ import re import tkinter as tk
    from    tkinter    import    ttk,
    messagebox

    # ====================================================================
    # Q1 LOGIC - Resume extraction
    # ====================================================================
    NAME_PATTERN       =       re.compile(r"^\s*(?:Name\s*[:\-]\s*)?([A-Z][a-z]+(?:\s+[A-Z][a-z]*\.?){1,3})\s*$",
    re.MULTILINE)
    EMAIL_PATTERN = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
    MOBILE_PATTERN      =      re.compile(r"(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{4}[\s-]?\d{5})(?!\d)")
    EXP_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b", re.IGNORECASE)
    SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
    SKILL_PATTERNS = {s: re.compile(r"\b" + re.escape(s) + r"\b", re.IGNORECASE) for s in SKILLS}

def build_profile(text):
        if  not  text  or  not  text.strip():  raise
            ValueError("Resume text is empty")
        m = NAME_PATTERN.search(text)  years = [float(y)  for
        y in EXP_PATTERN.findall(text)] return {
            "name": m.group(1).strip() if m else "Not found",
            "emails": EMAIL_PATTERN.findall(text),
            "mobiles": [re.sub(r"[\s-]", "", n) for n in MOBILE_PATTERN.findall(text)],
            "skills": [s for s, p in SKILL_PATTERNS.items() if p.search(text)],
            "experience": max(years) if years else 0.0,
        }

def is_eligible(profile, min_exp=2, skill="Python"):
        return profile["experience"] >= min_exp and skill in profile["skills"]


    SAMPLE_RESUMES = """Rahul Sharma
    Email: rahul.sharma@gmail.com
    Mobile: +91 98765 43210
    Skills: Python, SQL, Machine Learning, NLP
    Experience: 3 years as Data Scientist
    ---
    Name: Priya Nair
    Contact: priya.nair@outlook.com | 9123456780
    Technical Skills: Java, SQL, JavaScript
    Total experience of 5 years in backend development
    --- Arun Kumar
    arun_k@yahoo.co.i
    n Phone: 91234-
    56789
    Skills: python, nlp
    Fresher with 1 year internship experience
    ---   Sneha
    Reddy
    sneha.reddy@company.org
    +91-8012345678
    Expertise: Python, Java, Machine Learning
    Experience: 2.5 yrs in AI development"""

    # ====================================================================
    # Q2 LOGIC - Product search
    # ====================================================================
    PRODUCTS = [
        "Laptop", "Laptop Bag", "Gaming Laptop", "Wireless Mouse", "Mouse Pad",
        "Mechanical Keyboard", "Keyboard Cover", "Smartphone", "Smartwatch",
        "Phone Case", "Headphones", "Bluetooth Headphone", "Earphones",
        "USB Cable", "USB Charger", "Power Bank", "Laptop Stand", "Webcam",
        "Tablet", "Tablet Cover",
    ]

def search_products(keyword, mode, ignore_case):
        if not keyword.strip():
            raise ValueError("Search keyword cannot be empty")
        kw = re.escape(keyword.strip())
```

```
        pattern = {"Exact": rf"^{kw}$", "Prefix": rf"\b{kw}\w*",
                   "Suffix": rf"\w*{kw}\b", "Partial": kw}[mode]
        rx = re.compile(pattern, re.IGNORECASE if ignore_case else 0)
        return [p for p in PRODUCTS if rx.search(p)]


    # ======================================================================
    # Q3 LOGIC - Registration validation
    #     ======================================================================
    RULES = {
        "Register Number": (re.compile(r"19(?:2[0-5])\d{5}"), "9 digits, e.g. 192311234"),
        "Institutional Email": (re.compile(r"[A-Za-z0-9._%+-]+@(?:saveetha\.com|simats\.edu\.in)", re.I),
                                "must end with @saveetha.com or @simats.edu.in"),
        "Course Code": (re.compile(r"[A-Z]{3}\d{2}"), "3 capital letters + 2 digits, e.g. DSA03"),
        "Semester": (re.compile(r"[1-8]"), "a single digit from 1 to 8"),
        "Mobile Number": (re.compile(r"(?:\+91[\s-]?)?[6-9]\d{9}"), "10 digits starting with 6-9, optional +91"),
    }

def validate_field(field, value):
        pattern, hint = RULES[field]
        value = value.strip() if
        not value:
            return False, f"Field is empty ({hint})"
        if pattern.fullmatch(value): return
            True, "Valid"
        return False, f"Invalid - {hint}"


    # ======================================================================
    # GUI
    #     ======================================================================
    class App(tk.Tk):
        def __init__(self):
            super().__init__() self.title("DSA03 - NLP | CO1
            Assessment Tool 1") self.geometry("980x680")
            self.minsize(860, 600)
style = ttk.Style(self)
            style.theme_use("clam")
            style.configure("Title.TLabel", font=("Segoe UI", 15, "bold"),
            foreground="#1F3864") style.configure("Sub.TLabel", font=("Segoe UI", 10),
            foreground="#555555") style.configure("Accent.TButton", font=("Segoe UI", 10,
            "bold"), padding=6) style.configure("Treeview", rowheight=24, font=("Segoe UI",
            10)) style.configure("Treeview.Heading", font=("Segoe UI", 10, "bold"))
 ttk.Label(self, text="Natural Language Processing - Regular Expression Applications",
            style="Title.TLabel").pack(anchor="w", padx=14, pady=(12, 0))
            ttk.Label(self, text="SIMATS Engineering | Department of AI & Data Science",
                      style="Sub.TLabel").pack(anchor="w", padx=14, pady=(0, 8))
nb = ttk.Notebook(self) nb.pack(fill="both", expand=True, padx=12,
            pady=(0, 12)) self.build_resume_tab(nb)
            self.build_search_tab(nb) self.build_register_tab(nb)

                        ____________________________________
        #                                                   Tab 1
        ________________________________def build_resume_tab(self, nb):
            f = ttk.Frame(nb, padding=10) nb.add(f,
            text="1. Resume Extraction")
            ttk.Label(f, text="Paste resume text (separate multiple resumes with a line "
                              "containing ---):").pack(anchor="w")
            box = ttk.Frame(f) box.pack(fill="x", pady=4) self.resume_text =
            tk.Text(box, height=10, font=("Consolas", 10), wrap="word") sb =
            ttk.Scrollbar(box, command=self.resume_text.yview)
            self.resume_text.configure(yscrollcommand=sb.set)
            self.resume_text.pack(side="left", fill="x", expand=True)
            sb.pack(side="right", fill="y")
 bar = ttk.Frame(f) bar.pack(fill="x", pady=6) ttk.Button(bar, text="Load Sample Resumes",
            command=self.load_sample).pack(side="left") ttk.Button(bar, text="Extract
            Information", style="Accent.TButton",
                      command=self.extract_resumes).pack(side="left", padx=8)
            ttk.Button(bar, text="Clear", command=self.clear_resumes).pack(side="left")
            self.min_exp = tk.DoubleVar(value=2) ttk.Label(bar, text=" Min. experience
            (yrs):").pack(side="left", padx=(20, 2)) ttk.Spinbox(bar, from_=0, to=30,
            increment=0.5, width=5, textvariable=self.min_exp).pack(side="left")
cols = ("Name", "Email", "Mobile", "Skills", "Experience", "Eligible") widths =
            (120, 190, 100, 240, 80, 70) self.res_tree = ttk.Treeview(f,
```

```
            columns=cols,  show="headings",  height=7)  for  c,  w  in  zip(cols,
            widths):
                self.res_tree.heading(c,                          text=c)
                self.res_tree.column(c, width=w, anchor="w")
            self.res_tree.tag_configure("yes",
            background="#E2F4E2")
            self.res_tree.tag_configure("no",
            background="#FBE5E5")   self.res_tree.pack(fill="both",
            expand=True, pady=4)
 ttk.Label(f,  text="Shortlisted  candidates  (Python  +  minimum  experience):",
            font=("Segoe UI", 10, "bold")).pack(anchor="w", pady=(6, 0))
            self.short_label      =      ttk.Label(f,      text="-",      justify="left",      wraplength=900)
            self.short_label.pack(anchor="w")
def load_sample(self):
            self.resume_text.delete("1.0",                                "end")
            self.resume_text.insert("1.0", SAMPLE_RESUMES)
def clear_resumes(self):
            self.resume_text.delete("1.0", "end")
            self.res_tree.delete(*self.res_tree.get_children(
            )) self.short_label.config(text="-")
def extract_resumes(self):
            raw = self.resume_text.get("1.0", "end") blocks = [b for b in re.split(r"^\s*-{3,}\s*$", raw,
            flags=re.MULTILINE) if b.strip()] if not blocks: messagebox.showwarning("No input", "Please
            paste at least one resume or load the samples.") return
            self.res_tree.delete(*self.res_tree.get_children())
            try:
                min_exp = float(self.min_exp.get())
            except tk.TclError:
                messagebox.showerror("Invalid  value",  "Minimum  experience  must  be  a  number.")
                return
shortlisted  =  []  for  block  in
            blocks:
                p = build_profile(block) ok
                =  is_eligible(p,  min_exp)
                if ok:
                    shortlisted.append(p)
                self.res_tree.insert("", "end", tags=("yes" if ok else "no",), values=( p["name"],
                    ", ".join(p["emails"]) or "-", ", ".join(p["mobiles"]) or "-",
                    ", ".join(p["skills"]) or "-", f"{p['experience']:g} yrs", "YES" if ok else "NO"))
if shortlisted:
                lines = [f"* {p['name']} - {p['experience']:g} yrs - {', '.join(p['skills'])}"
                         for p in shortlisted]
                self.short_label.config(text="\n".join(lines) + f"\nTotal: {len(shortlisted)} of {len(blocks)}")
            else:
                self.short_label.config(text=f"No candidate meets the criteria (0 of {len(blocks)})")
        #                                                  Tab          2
                                    def build_search_tab(self, nb):
            f = ttk.Frame(nb, padding=10) nb.add(f,
            text="2. Product Search")
 bar   =   ttk.Frame(f)   bar.pack(fill="x")   ttk.Label(bar,
            text="Keyword:").pack(side="left")    self.kw    =
            tk.StringVar()      e      =      ttk.Entry(bar,
            textvariable=self.kw,                   width=24)
            e.pack(side="left", padx=6)
            e.bind("<Return>", lambda _e: self.run_search())
            ttk.Label(bar,   text="Search   type:").pack(side="left",   padx=(10,   0))   self.mode   =
            tk.StringVar(value="Partial")          ttk.Combobox(bar,          textvariable=self.mode,
            values=["Exact",         "Prefix",         "Suffix",         "Partial"],         width=10,
            state="readonly").pack(side="left", padx=6)
            self.ignore_case  =  tk.BooleanVar(value=True)  ttk.Checkbutton(bar,  text="Ignore  case",
            variable=self.ignore_case).pack(side="left",    padx=10)    ttk.Button(bar,    text="Search",
            style="Accent.TButton", command=self.run_search).pack(side="left")
            ttk.Button(bar, text="Clear Report", command=self.clear_report).pack(side="left", padx=6)
 mid  =  ttk.Frame(f)  mid.pack(fill="both",  expand=True,  pady=10)  left  =  ttk.LabelFrame(mid,
            text="Matching  products",  padding=6)  left.pack(side="left",  fill="both",  expand=True,
            padx=(0,    6))    self.result_list    =    tk.Listbox(left,    font=("Segoe    UI",    11),
            activestyle="none")  self.result_list.pack(fill="both",  expand=True)  self.count_label  =
            ttk.Label(left,    text="Total    matches:    -",    font=("Segoe    UI",    10,    "bold"))
            self.count_label.pack(anchor="w", pady=(6, 0))
  right    =    ttk.LabelFrame(mid,    text="Search    report",    padding=6)
            right.pack(side="left",  fill="both",  expand=True,  padx=(6,  0))
            cols  =  ("Keyword",  "Type",  "Case",  "Matches")  self.rep_tree  =
            ttk.Treeview(right,  columns=cols,  show="headings")  for  c,  w  in
            zip(cols, (140, 80, 100, 70)):
```

```
                self.rep_tree.heading(c,                        text=c)
                self.rep_tree.column(c, width=w)
            self.rep_tree.pack(fill="both", expand=True)
ttk.Label(f, text="All products: " + ", ".join(PRODUCTS), wraplength=900,
            style="Sub.TLabel").pack(anchor="w")
def run_search(self):
            try:
                results = search_products(self.kw.get(), self.mode.get(), self.ignore_case.get())
            except ValueError as err:
                messagebox.showerror("Invalid    search",    str(err))
                return
            self.result_list.delete(0,    "end")
            for       r       in       results:
            self.result_list.insert("end", r)
            if not results:
                self.result_list.insert("end", "No matching products found")
            self.count_label.config(text=f"Total matches: {len(results)}")
            self.rep_tree.insert("", "end", values=(
            self.kw.get().strip(), self.mode.get(),
                "Insensitive" if self.ignore_case.get() else "Sensitive", len(results)))
def clear_report(self):
            self.rep_tree.delete(*self.rep_tree.get_children())
            self.result_list.delete(0, "end")
            self.count_label.config(text="Total matches: -")

        #                                                  Tab          3
                                    def build_register_tab(self, nb):
            f = ttk.Frame(nb, padding=14)
            nb.add(f, text="3. Registration Validation")
  form   =   ttk.LabelFrame(f,   text="Student   registration   form",   padding=12)
            form.pack(fill="x") self.reg_vars, self.reg_msgs = {}, {}
            hints = {"Register Number": "192311234", "Institutional Email": "name@saveetha.com",
                     "Course Code": "DSA03", "Semester": "1 - 8", "Mobile Number": "+91 9876543210"}
            for i, field in enumerate(RULES):
                ttk.Label(form,  text=field  +  ":",  width=20).grid(row=i,  column=0,  sticky="w",
                pady=5)   var   =   tk.StringVar()   self.reg_vars[field]   =   var   ttk.Entry(form,
                textvariable=var,    width=30).grid(row=i,    column=1,    padx=8)    ttk.Label(form,
                text="e.g.  " + hints[field], style="Sub.TLabel", width=22).grid(row=i, column=2,
                sticky="w")
                msg  =  tk.Label(form,  text="",  anchor="w",  font=("Segoe  UI",  10,
                "bold"))     msg.grid(row=i,     column=3,     sticky="w",     padx=8)
                self.reg_msgs[field] = msg
bar = ttk.Frame(f) bar.pack(fill="x", pady=12) ttk.Button(bar, text="Validate
            &                Register",                style="Accent.TButton",
            command=self.validate_form).pack(side="left")
            ttk.Button(bar,      text="Fill      Valid      Sample",      command=lambda:
                       self.fill_sample(True)).pack(side="left", padx=8)
            ttk.Button(bar,    text="Fill    Invalid    Sample",    command=lambda:
                       self.fill_sample(False)).pack(side="left")
            ttk.Button(bar, text="Reset", command=self.reset_form).pack(side="left", padx=8)
 self.status  =  tk.Label(f,  text="REGISTRATION  STATUS:  -",  font=("Segoe  UI",  14,  "bold"),
            bg="#EEEEEE", fg="#333333", pady=14)
            self.status.pack(fill="x", pady=6) self.report = tk.Text(f, height=9, font=("Consolas",
            10), state="disabled", bg="#FAFAFA") self.report.pack(fill="both", expand=True)
def fill_sample(self, valid):
            data  =  (["192311234",  "rahul.s@saveetha.com",  "DSA03",  "5",  "+91  9876543210"]  if
            valid else ["12345", "rahul@gmail.com", "dsa3", "9", "1234567890"]) for var, val in
            zip(self.reg_vars.values(), data):
                var.set(val)
def reset_form(self):
            for     var     in     self.reg_vars.values():
                var.set("")
            for m in self.reg_msgs.values():
                m.config(text="")
            self.status.config(text="REGISTRATION      STATUS:      -",      bg="#EEEEEE",      fg="#333333")
            self._write_report("")
def _write_report(self, text):
            self.report.config(state="normal")
            self.report.delete("1.0", "end")
            self.report.insert("1.0", text)
            self.report.config(state="disabled")
def validate_form(self):
            lines, all_ok = ["FINAL REGISTRATION REPORT", "-" * 60], True
            for field, var in self.reg_vars.items():
```

```
            ok, msg = validate_field(field, var.get())
            all_ok &= ok
            short = "Valid" if ok else ("Empty" if not var.get().strip() else "Invalid
            format") self.reg_msgs[field].config(text=("\u2714 " if ok else "\u2718 ") +
            short, fg="#1B7F1B" if ok else "#C62828")
            lines.append(f"[{'PASS' if ok else 'FAIL'}] {field:<20}: {var.get().strip() or '(empty)'} ->
        {msg}") lines += ["-" * 60, "Result: " + ("Registration SUCCESSFUL" if all_ok else "Registration
        REJECTED - correct the errors")]
        self._write_report("\n".join(lines)) if all_ok: self.status.config(text="REGISTRATION
        STATUS: SUCCESSFUL", bg="#C8E6C9", fg="#1B5E20")
        else:
            self.status.config(text="REGISTRATION STATUS: REJECTED", bg="#FFCDD2", fg="#B71C1C")

if _name_== "_main_":
        App().mainloop()
```

Rubrics Evaluation (for Faculty Use)

Q. No.	Problem Context (20)	Application of Concepts (30)	Problem Solving (20)	Accuracy (20)	Clarity (10)	Sub Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____